MON APPRENTISSAGE — SERIES PYTHON

Python

Du zéro à l'expert

Chapitre 13 — Fonctions Avancées

**args, **kwargs, lambda, map, filter — le niveau supérieur*

Auteur :

KETOTSA AMÉVI CLAUDE

Date de publication :

11 mars 2026

Publié dans le cadre de mon parcours d'apprentissage Python

Table des matières

Chapitre 13 — Fonctions Avancées **2**
0.1 *args — Arguments positionnels variables 2
0.2 **kwargs — Arguments nommes variables 3
0.3 Les fonctions lambda 4
0.4 map() — Transformer une séquence 5
0.5 filter() — Filtrer une séquence 6
0.6 reduce() — Réduire une séquence 7
0.7 Les closures — Fonctions qui mémorisent 7
0.8 Les décorateurs — Enrichir les fonctions 9
0.9 Ce qui m'a le plus surpris dans ce chapitre 11
0.10 Résumé du Chapitre 13 11

Chapitre 13 — Fonctions Avancées

À retenir

Le chapitre précédent posait les bases des fonctions. Celui-ci va plus loin : `*args` et `**kwargs` pour accepter un nombre variable d'arguments, les fonctions `lambda` pour des expressions concises, `map()`, `filter()`, `reduce()` pour la programmation fonctionnelle, et les **closures** et **décorateurs** qui représentent l'élégance ultime de Python. Ce chapitre marque le passage vers la **maîtrise réelle** du langage.

*args — Arguments positionnels variables

`*args` permet à une fonction d'accepter un **nombre variable d'arguments positionnels**.

*args — nombre variable d'arguments

```python
# Sans *args      nombre fixe d'arguments
def additionner_3(a, b, c):
    return a + b + c

# Avec *args      nombre illimite d'arguments
def additionner(*args):
    print(type(args))    # <class 'tuple'>
    print(args)          # (1, 2, 3, 4, 5)
    return sum(args)

print(additionner(1, 2))            # 3
print(additionner(1, 2, 3))         # 6
print(additionner(1, 2, 3, 4, 5))   # 15

# Melanger parametres fixes et *args
def afficher_infos(titre, *elements):
    print(f"\n=== {titre} ===")
    for i, element in enumerate(elements, 1):
        print(f"  {i}. {element}")

afficher_infos("Fruits", "Pomme", "Banane", "Cerise")
afficher_infos("Langages", "Python", "JavaScript", "Rust", "Go"
    )

# Deballer une liste avec * lors de l'appel
nombres = [3, 7, 2, 9, 1]
print(additionner(*nombres))    # 22
```

À retenir

`*args` collecte tous les arguments positionnels supplémentaires dans un **tuple**. Le nom `args` est une convention — on pourrait écrire `*nombres` ou `*valeurs`. C'est l'étoile `*` qui fait la magie, pas le nom.

**kwargs — Arguments nommes variables

`**kwargs` permet d'accepter un **nombre variable d'arguments nommés**.

**kwargs — arguments nommés variables

```python
# **kwargs collecte les arguments nommes dans un DICT
def afficher_profil(**kwargs):
    print(type(kwargs))    # <class 'dict'>
    for cle, valeur in kwargs.items():
        print(f"  {cle:15} : {valeur}")

afficher_profil(prenom="Alice", age=25, ville="Paris")
afficher_profil(pseudo="bob42", score=1500, niveau="Expert")

# Melanger tout : fixes + *args + **kwargs
def fonction_complete(obligatoire, *args, **kwargs):
    print(f"Obligatoire : {obligatoire}")
    print(f"Args        : {args}")
    print(f"Kwargs      : {kwargs}")

fonction_complete(
    "indispensable",
    1, 2, 3,
    nom="Alice", ville="Paris"
)

# Cas pratique : construire une requete dynamique
def construire_requete(table, **conditions):
    where = " AND ".join(
        f"{k} = '{v}'" for k, v in conditions.items()
    )
    return f"SELECT * FROM {table} WHERE {where}"

requete = construire_requete(
    "utilisateurs",
    ville="Paris",
    age=25,
    actif=True
)
print(requete)
```

Ce qui m'a résisté

L'ordre des paramètres dans une définition de fonction est strict : **fixes → *args → kwargs avec défaut → **kwargs**. Écrire `def f(**kwargs, *args)` est une `SyntaxError`. J'ai mis du temps à mémoriser cet ordre — mais une fois compris, il est logique : on commence par ce qui est certain, on finit par ce qui est flexible.

Les fonctions lambda

Une **lambda** est une fonction anonyme définie en une seule expression.

Lambda — fonctions anonymes

```python
# Syntaxe : lambda parametres : expression

# Fonction classique
def doubler(x):
    return x * 2

# Equivalent en lambda
doubler_lambda = lambda x: x * 2

print(doubler(5))         # 10
print(doubler_lambda(5)) # 10

# Lambda avec plusieurs parametres
additionner = lambda a, b: a + b
print(additionner(3, 7))    # 10

puissance = lambda base, exp: base ** exp
print(puissance(2, 8))      # 256

# Lambda avec condition
absolu = lambda x: x if x >= 0 else -x
print(absolu(-15))    # 15
print(absolu(8))      # 8

# Lambda dans une expression directe
print((lambda x, y: x * y)(6, 7))    # 42
```

Lambda avec sorted(), min(), max()

```python
etudiants = [
    {"prenom": "Charlie", "note": 14},
    {"prenom": "Alice",   "note": 18},
    {"prenom": "Bob",     "note": 11},
    {"prenom": "Diana",   "note": 16},
]
```

```python
# Trier par note avec lambda comme key
par_note = sorted(etudiants, key=lambda e: e["note"])
for e in par_note:
    print(f"{e['prenom']:10} : {e['note']}")

# Trier par prenom
par_prenom = sorted(etudiants, key=lambda e: e["prenom"])

# Meilleur etudiant
meilleur = max(etudiants, key=lambda e: e["note"])
print(f"\nMeilleur : {meilleur['prenom']} ({meilleur['note']})"
    )

# Trier des tuples par 2e element
points = [("Alice", 85), ("Bob", 92), ("Charlie", 78)]
points.sort(key=lambda x: x[1], reverse=True)
print(points)
```

Note importante

Les lambdas sont idéales pour des **fonctions courtes et ponctuelles** — notamment comme argument `key` de `sorted()`, `min()`, `max()`. Pour des fonctions plus complexes ou réutilisables, préférer toujours `def`. Une lambda illisible est pire qu'une fonction nommée claire.

map() — Transformer une séquence

`map()` applique une fonction à **chaque élément** d'une séquence.

map() — appliquer une fonction à chaque élément

```python
# map(fonction, iterable)      retourne un objet map
nombres = [1, 2, 3, 4, 5]

# Avec une fonction nommee
def au_carre(x):
    return x ** 2

carres = list(map(au_carre, nombres))
print(carres)    # [1, 4, 9, 16, 25]

# Avec lambda      plus concis
carres = list(map(lambda x: x ** 2, nombres))
print(carres)    # [1, 4, 9, 16, 25]

# Convertir des types
chaines = ["1", "2", "3", "4", "5"]
entiers = list(map(int, chaines))
```

```python
print(entiers)   # [1, 2, 3, 4, 5]

# map() avec plusieurs listes
a = [1, 2, 3]
b = [10, 20, 30]
sommes = list(map(lambda x, y: x + y, a, b))
print(sommes)   # [11, 22, 33]

# Cas pratique
prenoms = ["alice", "bob", "charlie"]
capitalises = list(map(str.capitalize, prenoms))
print(capitalises)   # ['Alice', 'Bob', 'Charlie']

# Comparaison avec list comprehension
# map()              : list(map(lambda x: x**2, nombres))
# list comprehension: [x**2 for x in nombres]
# Les deux sont valides     list comprehension est plus lisible
```

filter() — Filtrer une séquence

`filter()` retourne uniquement les éléments pour lesquels la fonction retourne `True`.

filter() — filtrer selon une condition

```python
nombres = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Garder seulement les pairs
pairs = list(filter(lambda x: x % 2 == 0, nombres))
print(pairs)   # [2, 4, 6, 8, 10]

# Garder les positifs
valeurs = [-3, 5, -1, 8, -7, 2, 0, 4]
positifs = list(filter(lambda x: x > 0, valeurs))
print(positifs)   # [5, 8, 2, 4]

# Filtrer des chaines non vides
mots = ["Python", "", "est", "", "g nial", ""]
non_vides = list(filter(None, mots))   # None = filter les
    falsy
print(non_vides)   # ['Python', 'est', 'g nial']

# Cas pratique : filtrer des etudiants admis
etudiants = [
    {"prenom": "Alice",   "note": 15},
    {"prenom": "Bob",     "note": 8},
    {"prenom": "Charlie", "note": 12},
    {"prenom": "Diana",   "note": 4},
    {"prenom": "Eve",     "note": 17},
]
```

```python

admis = list(filter(lambda e: e["note"] >= 10, etudiants))
for e in admis:
    print(f"      {e['prenom']}      {e['note']}/20")
```

reduce() — Réduire une séquence

`reduce()` combine tous les éléments d'une séquence en **une seule valeur**.

reduce() — réduire à une valeur

```python
from functools import reduce

nombres = [1, 2, 3, 4, 5]

# Calculer le produit de tous les elements
produit = reduce(lambda acc, x: acc * x, nombres)
print(produit)   # 120  (1     2345     )

# Maximum sans max()
maximum = reduce(lambda a, b: a if a > b else b, nombres)
print(maximum)   # 5

# Concatener une liste de chaines
mots = ["Python", " ", "est", " ", "incroyable"]
phrase = reduce(lambda a, b: a + b, mots)
print(phrase)   # Python est incroyable

# Avec valeur initiale (3e argument)
somme = reduce(lambda acc, x: acc + x, nombres, 100)
print(somme)   # 115  (100 + 1+2+3+4+5)

# Visualiser le fonctionnement pas a pas
def tracer_reduce(acc, x):
    resultat = acc + x
    print(f"  {acc} + {x} = {resultat}")
    return resultat

print("Etapes de reduce :")
reduce(tracer_reduce, [1, 2, 3, 4, 5])
```

Les closures — Fonctions qui mémorisent

Une **closure** est une fonction imbriquée qui **mémorise** les variables de sa fonction parente, même après que celle-ci ait terminé.

Closures — mémoriser le contexte

```python
# La fonction externe cree un contexte
def creer_multiplicateur(facteur):
    # La fonction interne capture 'facteur'
    def multiplier(x):
        return x * facteur    # 'facteur' est memoris  !
    return multiplier    # on retourne la fonction

# Creer des multiplicateurs specialises
doubler  = creer_multiplicateur(2)
tripler  = creer_multiplicateur(3)
decupler = creer_multiplicateur(10)

print(doubler(5))     # 10
print(tripler(5))     # 15
print(decupler(5))    # 50

# Verifier que 'facteur' est bien memorise
print(doubler.__closure__[0].cell_contents)    # 2
print(tripler.__closure__[0].cell_contents)    # 3

# Exemple pratique : compteur avec closure
def creer_compteur(debut=0):
    compte = [debut]    # liste pour contourner nonlocal

    def incrementer(pas=1):
        compte[0] += pas
        return compte[0]

    def reinitialiser():
        compte[0] = debut

    def valeur():
        return compte[0]

    return incrementer, reinitialiser, valeur

inc, reset, val = creer_compteur(10)
print(inc())       # 11
print(inc())       # 12
print(inc(5))      # 17
print(val())       # 17
reset()
print(val())       # 10
```

Ce qui m'a résisté

Les closures m'ont d'abord semblé presque magiques — comment une fonction peut-elle "se souvenir" de variables d'une autre fonction qui a déjà terminé son exécution ? La réponse est que Python maintient une référence à l'environnement de la fonction externe dans un objet `__closure__`. Ce concept est fondamental pour comprendre les décorateurs et la programmation fonctionnelle avancée. Il m'a fallu plusieurs exemples concrets avant que ça devienne intuitif.

Les décorateurs — Enrichir les fonctions

Un **décorateur** est une fonction qui prend une autre fonction en entrée et retourne une version **enrichie** de cette fonction.

Décorateurs — syntaxe de base

```python
# Un decorateur est une fonction qui enveloppe une autre
def mon_decorateur(fonction):
    def wrapper(*args, **kwargs):
        print("    Avant l'execution")
        resultat = fonction(*args, **kwargs)
        print("    Apres l'execution")
        return resultat
    return wrapper

# Sans sucre syntaxique
def saluer(prenom):
    print(f"Bonjour, {prenom} !")

saluer_decoree = mon_decorateur(saluer)
saluer_decoree("Alice")

# Avec le sucre syntaxique @     equivalent et elegant
@mon_decorateur
def saluer(prenom):
    print(f"Bonjour, {prenom} !")

saluer("Bob")
#         Avant l'execution
#     Bonjour, Bob !
#         Apres l'execution
```

Décorateurs pratiques

```python
import time
from functools import wraps

# Decorateur de chronometrage
def chronometrer(fonction):
```

```python
    @wraps(fonction)
    def wrapper(*args, **kwargs):
        debut = time.time()
        resultat = fonction(*args, **kwargs)
        fin = time.time()
        duree = fin - debut
        print(f"    {fonction.__name__} : {duree:.4f}s")
        return resultat
    return wrapper

# Decorateur de validation
def valider_positif(fonction):
    @wraps(fonction)
    def wrapper(n):
        if n < 0:
            raise ValueError(f"Argument doit etre positif, recu
                : {n}")
        return fonction(n)
    return wrapper

# Decorateur de logging
def logger(fonction):
    @wraps(fonction)
    def wrapper(*args, **kwargs):
        print(f"       Appel : {fonction.__name__}({args}, {
            kwargs})")
        resultat = fonction(*args, **kwargs)
        print(f"    R sultat : {resultat}")
        return resultat
    return wrapper

# Utilisation combinee de plusieurs decorateurs
@chronometrer
@logger
@valider_positif
def factorielle(n):
    if n == 0:
        return 1
    return n * factorielle(n - 1)

print(factorielle(5))
```

Ce qui m'a résisté

Deux subtilités majeures sur les décorateurs :

1. `@wraps(fonction)` est indispensable. Sans lui, la fonction décorée perd son nom et sa docstring — `fonction.__name__` retourne `"wrapper"` au lieu du vrai nom. `@wraps` préserve les métadonnées de la fonction originale.

2. L'ordre des décorateurs. Quand on empile plusieurs `@`, ils s'appliquent **de bas en haut**. `@A` puis `@B` sur une fonction `f` donne `A(B(f))`. L'ordre peut changer radicalement le comportement — comprendre cette pile est essentiel.

Ce qui m'a le plus surpris dans ce chapitre

Ce qui m'a résisté

Trois révélations de ce chapitre :

1. `*args` et `**kwargs` — déballer avec `*` et `**`. Je savais que `*args` collectait des arguments. J'ai découvert qu'on pouvait aussi **déballer** une liste ou un dict lors d'un appel : `f(*ma_liste)` et `f(**mon_dict)`. C'est le même opérateur, dans le sens inverse. Cette symétrie — collecter ET déballer — est l'une des syntaxes les plus puissantes et les plus élégantes de Python.

2. Les closures défient notre intuition de la durée de vie des variables. Normalement, quand une fonction se termine, ses variables locales disparaissent. Avec une closure, elles **survivent** — encapsulées dans la fonction retournée. C'est un mécanisme de persistance d'état sans classe, sans variable globale. Élégant, mais déroutant jusqu'à ce qu'on comprenne comment Python gère la mémoire des objets référencés.

3. Les décorateurs sont des closures déguisées. En décomposant un décorateur, j'ai réalisé que c'est simplement une closure qui prend une *fonction* comme variable capturée. Tout s'est connecté : les fonctions comme objets (chapitre 12), les closures, et les décorateurs ne sont qu'une seule et même idée déclinée à trois niveaux de sophistication. Python est cohérent jusque dans ses concepts les plus avancés.

Résumé du Chapitre 13

Ce que j'ai appris dans ce chapitre :

- `*args` — nombre variable d'arguments positionnels (tuple).
- `**kwargs` — nombre variable d'arguments nommés (dict).
- `lambda` — fonction anonyme en une expression, idéale comme `key=`.
- `map()` — appliquer une fonction à chaque élément.
- `filter()` — garder uniquement les éléments qui satisfont une condition.
- `reduce()` — réduire une séquence à une valeur unique.
- **Closures** — fonctions qui mémorisent leur contexte de création.
- **Décorateurs** `@` — enrichir une fonction sans modifier son code.
- `@wraps` — préserver les métadonnées de la fonction décorée.

"Les fonctions avancées ne sont pas de la complexité pour la complexité. Ce sont des outils pour écrire moins, exprimer plus, et penser mieux."